

3 IRC

Mise en œuvre d'un système à microprocesseur

Quelques rappels

1. Objectifs globaux du module :

- Séance 1 : Prise en main de Microvision. Premiers codes en C
- Séance 2 : Programmation des entrées-sorties.
- Séance 3 : Mise en œuvre d'interruptions
- Séance 4 : Mise en œuvre de Timer
- Séance 5 : Mise en œuvre d'UART
- Séance 6 : Mise en œuvre de périphériques analogiques DAC et ADC
- Séance 7 : Evaluation individuelle - Mise en œuvre d'un périphérique

2. Documentations disponibles sur e-campus :

- Supports de cours
- Initiation à Microvision
- Documentation Microcontrôleur 8051
- Architecture et outils de développement
- Datasheet 8051F020
- 8051 Instruction Set
- Architecture 8051
- Langage C et microcontrôleur 8051

3. Organisation des TP

Les TP se font en binômes, voire en trinômes (selon les contraintes de salle)

4. Répertoire de travail :

Si vous utilisez les PC de l'école, travaillez de préférence en local. Le projet μ Vision devra être enregistré sur le disque dur du PC. Placez le projet dans le dossier :

C:\CPE_users\TP_IRC\TPn\noms_étudiants

Si vous utilisez votre propre PC, vous êtes libres d'organiser votre arborescence comme vous le souhaitez.

5. Evaluation

La note finale de TP correspond à la note obtenue lors de **l'évaluation individuelle lors de la dernière séance**. Un **bonus** pourra être accordé en fonction de l'investissement observé lors des séances de travaux pratiques.

TP1 - Prise en main de Microvision - Premiers codes en C**1. Objectifs de la séance**

- **Connaissance de l'environnement de développement Microvision5**

- Ouverture d'un projet Microvision
- Compilation d'un code existant
- Simulation
- Prise en main de la carte d'évaluation
- Exécution sur la carte d'évaluation
- Modification du code existant.

- **Exercices basiques de codage en C**

2. Compétences acquises en fin de séance

- **A l'issue de cette première séance, vous devriez être capable de :**

- Prendre en main l'environnement de développement Keil μ Vision 5 pour une cible microcontrôleur 8051
- Ouvrir (créer) et configurer un projet de développement embarqué sous μ Vision
- Compiler et analyser un code C existant, identifier et corriger des erreurs de compilation et de liaison
- Utiliser les outils de simulation et de débogage (exécution pas à pas, points d'arrêt, observation des registres et variables)
- Programmer et exécuter une application sur une carte d'évaluation microcontrôleur
- Modifier et adapter un code C existant pour répondre à un comportement fonctionnel attendu
- Mettre en œuvre des structures de programmation C simples (variables, boucles, fonctions) dans un contexte embarqué

3. Activités en séance

3.1. ACTIVITE 1 -- Prise en main en main de l'environnement de développement Microvision au travers de l'utilisation d'un projet existant.

Télécharger depuis le E-campus le projet *Projet_TP1*. Le placer dans votre répertoire de travail : `C:\CPE_users\TP_IRC\TPn\noms_étudiants` et le décompresser.

Nous allons procéder aux opérations ci-dessous afin de s'assurer que le projet s'exécute correctement. On ne s'occupe pas encore de ce que fait le programme mais on veut s'assurer que la boucle infinie de la fonction *main* s'exécute en continu.

- Ouverture du projet sous Microvision.
- Première compilation.
- Test du code avec une exécution en mode simulation.
- Exécution en mode pas à pas (différentes options dans le pas à pas)
- Exécution avec des points d'arrêt. Pose d'un point d'arrêt, choix du positionnement
- Visualisation de la fenêtre de désassemblage
- Constat ?
- Conclure.

- **Insérer dans le projet le fichier de librairie TP1_lib_C**
- Tester en simulation
- Constat ?

- **Insérer dans le projet le fichier TP1_Lib_ASM.**
- Tester de nouveau et vérifier que la boucle infinie s'exécute... de manière infinie.

- Analyse de l'assembleur généré dans la fonction *Oscillator_init*. Quelles sont les adresses mémoire de la variable *i*. combien de lignes de code assembleur la ligne `for(...)` occupe-t-elle ? Idem pour le `while`. Expliquez le code assembleur généré pour cette ligne.

- Brancher la carte et le boîtier de débogage.
- Changer le mode de débogage et passer en « exécution 8081F020 TB » ; Recompiler

- Tester l'exécution sur la carte. Le seul signe visible de l'exécution du code est le clignotement de la LED. A l'aide du débogueur, déterminer les 2 lignes de codes qui provoquent l'allumage et l'extinction de la LED

3.2. ACTIVITE 2 – Premiers codes en C (Utilisation des variables de type `sfr`, `sfr16`, `bit` et `sbit`)

- On souhaite utiliser le port P3 en le nommant `Port_OUT`. Faites la déclaration qui le permet et testez-la.
- Toujours dans le port 3, on cherche à nommer la broche P3.0 « `OUT_P30` » et P3.2 « `OUT_P32` ». Faites les déclarations qui le permettent et testez.

- Déclarez une variable booléenne nommée « Test_bit ». Faites-la changer d'état dans la boucle infinie de la fonction *main*. Et indiquez où elle est logée en mémoire.
- Est-il possible de déclarer une variable sfr16 que nous nommerions T0 pour concaténer les registres TLO et TH0 ? Justifiez.
- Remplacez les 2 lignes de code qui changent l'état de la LED par un code qui utilise une déclaration de type sbit.
- Est-il possible de manipuler le bit 3 du registre P4 avec une déclaration sbit ? pourquoi ?

3.3. ACTIVITE 3 – choix du type de variable (Mesurer les temps d'exécution d'un programme en fonction des types de variables utilisés)

- En reprenant l'exemple du cours sur les variables permettant de mémoriser une durée. Tester le temps d'exécution du code pour les 3 cas suivants : incrémentation de 0,2s sur les variables seconde, milliseconde et centième-de-seconde initialisées à 2,2s.
- Conclure.
- Incrémentez une seconde fois les variables de 0,2s. Que pouvez-vous observer ?

3.4. ACTIVITE 4 – Opérations bit à bit – Masquages

- Mettre à 1, le bit 4 de P3 et à 0 le bit 5 de P3 sans modifier les autres bits de P3.
- Complémenter le bit 6 de P3 sans modifier les autres bits de P3.
- Mettre à 1, le bit 4 de P5 et à 0 le bit 5 de P5 sans modifier les autres bits de P5
- Complémenter le bit 6 de P4 sans modifier les autres bits de P4.
- Ecrire une boucle d'attente: tant que le bit 1 du registre SCON0 est à zéro, alors attendre...
- Ecrire une boucle d'attente: tant que le bit 1 du registre SCON1 est à zéro, alors attendre...

3.5. ACTIVITE 5 – Passage de paramètres aux fonctions.

- Nous avons mis à disposition un programme assembleur nommé *tempo_sup* qui permet de générer une temporisation selon 2 paramètres passés :
 - Un code ASCII qui correspond à l'unité de la valeur transmise ('S' seconde , 'M' milliseconde et 'U' microseconde pour fixer le multiplicateur)
 - Une valeur qui fixe la valeur de la temporisation à générer en tenant compte du multiplicateur transmis.
- Compte tenu des règles de passage de paramètres, écrivez le prototype de la fonction.
- Testez cette fonction dans la boucle infinie de la fonction *main*.
- Concluez.

3.6. ACTIVITE 6 –Facultative – Lectures-Ecritures dans la mémoire - Pointeurs

- Ecrire une fonction de recopie d'une chaîne de caractère dans une zone de mémoire vive. La chaîne de caractères source fera environ 50 caractères (la chaîne source sera une chaîne constante) et la zone de mémoire destination fera une centaine d'octets. Ecrivez 2 versions de code, une version utilisant des pointeurs génériques et une version avec des pointeurs spécifiques.
- Comparez les durées d'exécution.